

9

Setting Up a Backend for a Game Using Azure Services

In this chapter, we will set up a backend for a game app with real-time communication. We will not only create a backend that can scale up to handle a large number of users but also scale down when the number of users is reduced. To build that backend, we will use a serverless architecture based on services in **Microsoft Azure**.

The following topics will be covered in this chapter:

- Understanding the different Azure serverless services
- Creating a **SignalR** service in Microsoft Azure
- Using Azure Functions as an **application programming interface (API)**

Technical requirements

To be able to complete this project, you need to have Visual Studio for Mac or PC installed, as well as the necessary .NET MAUI components. See *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment.

You also need an Azure account. If you have a Visual Studio subscription, there are a specific amount of Azure credits included each month. To activate your Azure benefits, go to <https://my.visualstudio.com>.

You can also create a free account, where you can use selected services for free over 12 months. You will get \$200 worth of credit to explore any Azure service for 30 days, and you can also use the free services at any time. Read more at <https://azure.microsoft.com/en-us/free/>.

If you do not have and do not want to sign up for a free Azure account, you can use local development tools to run the services without Azure.

You can find the full source for the code in this chapter at <https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition>.

Project overview

The main aim of this project will be to set up the backend for a game. A large part of the project will be the configuration that we will carry out in the Azure portal. We will also write some code for the Azure functions

that will handle the SignalR connections and a bit of the game logic and state. SignalR is a library that makes real-time communication in applications easier. Azure SignalR is a service that makes it easier to connect multiple clients to send messages via the SignalR library. SignalR is described in more detail later. There will be functions to return information about the SignalR connection, manage matching players to play against each other, and post the result of each player's turn to the SignalR service.

The following diagram shows an overview of the architecture of this application:

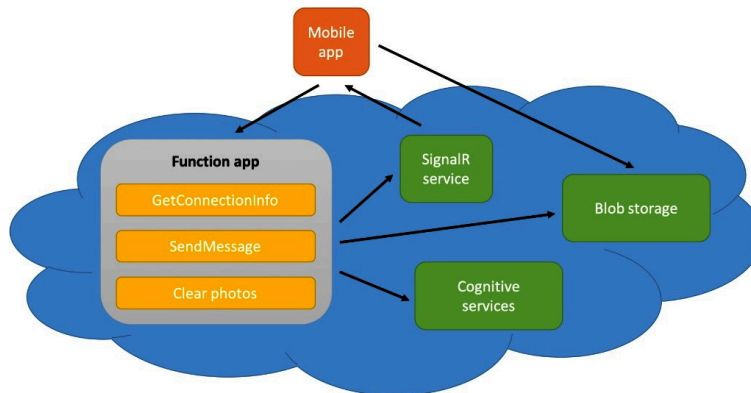


Figure 9.1 – Application architecture

The estimated time to complete this part of the project is about 2 hours.

An overview of the game

Sticks & Stones is a turn-based social game based on the concepts of two childhood games mashed into one, Dots and Boxes and Tic-Tac-Toe. The game board is laid out in a 9x9 grid. Each player will take a turn placing a stick along the side of a box, earning one point. If a stick completes a box, then the player takes ownership of the box, earning five points. The game is won when a player owns three boxes in a row, horizontally, vertically, or diagonally. If no player can own three boxes in a row, the winner of the game is determined by the player with the highest score.

To keep the app and the service side relatively simple, we will eliminate a lot of state management. When the player opens the app, they will have to connect to the game service. They will have to provide a gamer tag or username and an email address. Optionally, they can upload a picture of themselves to use as a profile picture.

Once connected, the player will see a list of all the other players connected to the same game service; this is called the lobby. The player's status, either "Ready to play" or "In a match," will be displayed along with the player's gamer tag and profile picture. If the player is not in a match, then there will also be a button to challenge the player to a match.

Challenging a player to a match will cause the app to prompt the opponent to respond to the challenge, either accept or decline. If the opponent accepts the challenge, then both players are navigated to a new game board where the player who received the challenge will have the first

turn. Both players' statuses will update to "In a match" in all the other players' lobbies. Play will alternate between players as they choose a location to place a single stick. Each time a stick is placed by a player, the game board and score will update on both players' devices. When a stick is placed that completes one or more squares, the player then "owns" that square, and a pile of stones is placed in the center of the square. When all sticks have been placed, or a player owns three stones in a row, the game is over, the players navigate back to the lobby, and their status is updated to "Ready to play."

If a player leaves the app during a game, then they will have forfeited the game and the remaining opponent will be credited with the win and navigated back to the lobby.

The following screenshot should give you an idea of what the app will look like when it's completed in *Chapter 10*:

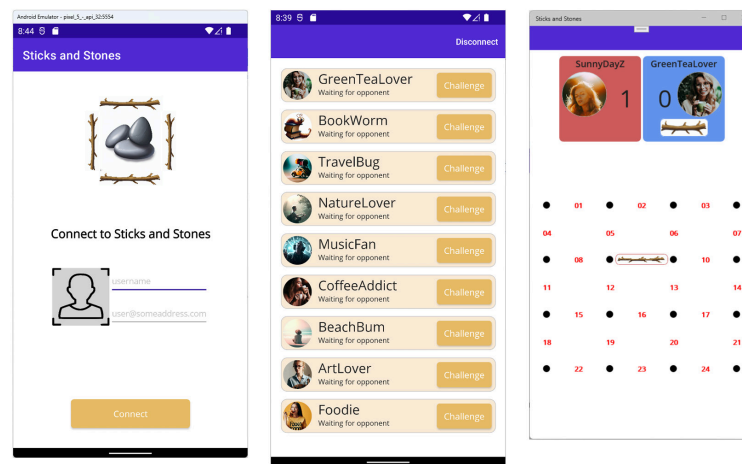


Figure 9.2 – The main game screens

Understanding the different Azure serverless services

Before we start to build a backend with a serverless architecture, we need to define what **serverless** means. In a serverless architecture, the code will run on a server, but we don't need to worry about that; the only thing we need to focus on is building our software. We let someone else handle everything to do with servers. We don't need to think about how much memory or **central processing units (CPUs)** the server needs, or even how many servers we need. When we use services in Azure, Microsoft takes care of this for us.

Azure SignalR Service

Azure SignalR Service is a service in **Microsoft Azure** for real-time communication between a server and clients. The service will push content to the clients without them having to poll the server to get content updates. SignalR can be used for multiple types of applications, including mobile applications, web applications, and desktop applications.

SignalR will use **WebSockets** if that option is available. If it is not, SignalR will use other techniques for communication, such as **Server-Sent Events (SSEs)** or **long polling**. SignalR will detect which transport technology is available and use it without the developer having to think about it at all.

SignalR can be used in the following examples:

- **Chat applications:** Where the application requires updates from the server as soon as new messages are available
- **Collaborative applications:** For example, meeting applications or when users on multiple devices are working with the same document
- **Multiplayer games:** Where all users need live updates about other users
- **Dashboard applications:** Where users need live updates

Azure Functions

Azure Functions is a Microsoft Azure service that allows us to run code in a serverless way. We will deploy small pieces of code called **functions**. Functions are deployed in groups, called **function apps**. When we are creating a function app, we need to select whether we want it to run on a Consumption plan or an App Service plan. We select a Consumption plan if we want the application to be completely serverless, while with an App Service plan, we have to specify the requirements of the server. With a Consumption plan, we pay for the execution time and for how much memory the function uses. One benefit of an App Service plan is that you can configure it to be **Always-On**, and you won't have any cold starts, so long as you don't have to scale up to more instances. The big benefit of a Consumption plan is that it will always scale according to which resources are needed at that time.

There are several ways in which a function can be triggered to run. Two examples are **HttpTrigger** and **TimerTrigger**. **HttpTrigger** will trigger the function to run when an HTTP request is calling the function. With **TimerTrigger**, functions will run at an interval that we specify. There are also triggers for other Azure services. For example, we can configure a function to run when a file is uploaded to Azure Blob storage, when a new message is posted to an event hub or service bus, or when data is changed in an Azure Cosmos DB service.

Now that we understand what features Azure SignalR Service and Functions offer, let's use them to build our game backend.

Building the serverless backend

In this section, we will set up the backend based on the services described in the preceding section.

Creating a SignalR service

The first service that we will set up is the one for SignalR. To create such a service, proceed as follows:

1. Go to the Azure portal at <https://portal.azure.com>.
2. Create a new resource. The **SignalR Service** resource is in the **Web & Mobile** category.
3. Provide a name for the resource in the form.
4. Select the subscription you want to use for this project.
We recommend that you create a new **Resource group** and use it for all the resources that we will create for this project. The reason that we want one resource group is that it is easier to track which resources are related to this project, and it is also easier to delete all the resources together.
5. Select a location that is close to your users.
6. Select a pricing tier. For this project, we will use the **Free** tier. We can always use the **Free** tier for development and later scale up to a tier that can handle more connections.
7. Set **Service mode** to **Serverless**.
8. Click **Review + create** to review the settings before creating the SignalR service.
9. Click **Create** to create the storage account.

Refer to the following screenshot to view the preceding information:

SignalR

Deploy fully managed SignalR Service at scale. [Learn more about SignalR](#)

Project Details

Subscription * MSDN Subscription

Resource group * DOTNET-MAUI-Projects

Service Details

Resource Name * SticksAndStones

Region * East US

Pricing tier * Free

Service mode Serverless

Figure 9.3 – Creating the SignalR service

This is all we need to do to set up a SignalR service. We will return to it in the Azure portal later to grab a connection string to it.

The next step is to set up a storage account in which we can store the images that are uploaded by the users.

With the Computer Vision service created, we can now create the Azure Functions service, which will run our game logic and use SignalR, Blob storage, and Cognitive Services, which we just created.

Using Azure Functions as an API

All the code we will write for the backend will be in Azure Functions. We will use a Visual Studio project to write, debug, and deploy our functions. Before we create the project, we will have to set up and configure the Azure Functions service. Then, we will implement the function to connect the player to the game and provide the client with a list of current players. Next, we will write the functions that allow one player to challenge another to a game. Finally, we will wrap up by writing the function that allows players to take turns placing sticks on the board.

Let's begin by creating the Azure Functions service.

Creating the Azure Functions service

Before we write any code, we will create the function app. This will contain the functions in the Azure portal. Proceed as follows:

1. Create a new **Function App** resource. **Function App** can be found under the **Compute** category.
2. Select a subscription for the function app.
3. Select a resource group for the function app. This should be the same as the other resources we have created in this chapter.
4. Give the function app a name. The name will also be the start of the URL of the function.
5. Select **Code** as the deployment mechanism.
6. Select **.NET** as the runtime stack for the functions.
7. Select **.NET 6.0 (Long Term Support)** for the version.
8. Select a location that is closest to your users.
9. Select **Windows** for **Operating System**.
10. We will use the **Consumption** plan as our **Hosting** plan, so we'll only pay for what we use. **Function app** will scale both up and down according to our requirements – without us having to think about it at all – if we select a **Consumption** plan.

Refer to the following screenshot to view the preceding information:

Create Function App

Basics Storage Networking Monitoring Deployment Tags Review + create

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ MSDN Subscription

Resource Group * ⓘ DOTNET-MAUI-Projects

[Create new](#)

Instance Details

Function App name * SticksAndStones ✓
.azurewebsites.net

Do you want to deploy code or container image? * ☒ Code ☐ Container Image

Runtime stack * .NET

Version * 6 (LTS)

Region * East US

Operating system

The Operating System has been recommended for you based on your selection of runtime stack.

Operating System * ☐ Linux ☒ Windows

Hosting

The plan you choose dictates how your app scales, what features are enabled, and how it is priced. [Learn more](#)

Hosting options and plans * ⓘ

☒ Consumption (Serverless)
Optimized for serverless and event-driven workloads.

☐ Functions Premium
Event based scaling and network isolation, ideal for workloads running continuously.

☐ App service plan
Fully isolated and dedicated environment suitable for workloads that need large SKUs or need to co-locate Web Apps and Functions.

Figure 9.4 – Create Function App – Basics

11. Click **Review + create** to review the settings before creating the function app.
12. Click **Create** to create the function app.

Creating the projects

If you want, you can create functions in the Azure portal. I prefer to use Visual Studio, however, because the code editing experience is better, and you can use source control. For this project, we will need to separate projects as part of our solution – an Azure Functions project and a class library for shared code between the functions and the .NET MAUI app that will be built in *Chapter 10*. To create and configure the projects, proceed as follows:

1. Create a new project in Visual Studio.
2. Enter **function** in the search field to find the template for Azure Functions.
3. Click the **Azure Functions** template to continue, as illustrated in the following screenshot:

Create Function App ...

Basics **Storage** Networking Monitoring Deployment Tags Review + create

Storage

When creating a function app, you must create or link to a general-purpose Azure Storage account that supports Blobs, Queue, and Table storage.

Storage account *

sticksandstones (v2) ▼

Create new

Figure 9.5 – Create a new project

- 4. Name the project **SticksAndStones.Functions**.
- 5. Name the solution **SticksAndStones.Functions**, as illustrated in the following screenshot, and click **Next**:

Configure your new project

Azure Functions C# Azure Cloud

Project name

SticksAndStones.Functions

Location

C:\Users\cummings.michael\Source\Repos

...

Solution name ?

SticksAndStones

☐ Place solution and project in the same directory

Project will be created in "C:\Users\cummings.michael\Source\Repos\SticksAndStones\SticksAndStones.Functions\"

BackNext

Figure 9.6 – Configure your new project

The next step is to create our first function, as follows:

- 1. Select **.Net 6.0 (Long Term Support)** for **Functions worker** at the top of the dialog box.
- 2. Select **Http trigger** as the trigger for our first function.
- 3. Click **Create** to continue; our functions project will be created.

Refer to the following screenshot to view the preceding information:

Additional information

Azure Functions C# Azure Cloud

Functions worker ?

.NET 6.0 (Long Term Support)

Function ?

Http trigger

☒ Use Azurite for runtime storage account (AzureWebJobsStorage) ?

☐ Enable Docker ?

Authorization level ?

Anonymous

BackCreate

Figure 9.7 – Creating a new Azure Functions application – Additional information

Our first function will return the connection information for the SignalR service. To do that, we need to connect the function by adding a connection string to the SignalR service, as follows:

1. Go to the **SignalR Service** resource in the Azure portal.
2. Go to the **Keys** tab on the left and copy the connection string.
3. Go to the **Function App** resource and add the connection string under **Application Settings**. Use **AzureSignalRConnectionString** as the name for the setting.
4. Add the connection string to the **Values** array in the **local.settings.json** file in the Visual Studio project to be able to run the function locally on the development machine, as illustrated in the following code block:

```
{ "IsEncrypted": false, "Values": { "AzureWebJobsStorage": "UseDevelopmentStorage=true",
```

If the file doesn't exist in your project, create a new file in the root of the **SticksAndStones.Functions** project, and add the code listed previously with your connection string.

Next, in the **SticksAndStones.Functions** project, we need to reference the **Microsoft.Azure.WebJobs.Extensions.SignalRService** NuGet package. This package contains the classes we need to communicate with the SignalR service. If an error occurs during this and you are not able to install the package, make sure that you have the latest version of all other packages in the project and try again.

The last change we need to make is to adjust the automatic namespace generation. By default, the default namespace is the name of the project, which means all types in this project would have a root namespace of **SticksAndStones.Functions**. We don't need the **Functions** part of that, so let's remove it:

1. Right-click the **SticksAndStones.Functions** project in **Solution Explorer** and select **Properties**.
2. In the **Properties** window, use the search box at the top to search for **Default namespace**.
3. Change the **Default namespace** property value to **\$(MSBuildProjectName.Split(".")[0].Replace(" ", "_"))**.
This will split the project name on ., using only the first part and replacing any spaces with underscores.

Now, when we create a new class, the namespace will start with just **SticksAndStones**. It's time to create a shared project so that we can reuse code in both the .NET MAUI client and the Azure Functions service.

The shared code will go into a class library project. To create the project and reference it from the **SticksAndStones.Functions** project, follow these steps:

1. Right-click on the **SticksAndStones** solution node in **Solution Explorer** and select **Add**, then **New Project**.
2. Search for **Class Library** in the **Add a new project** dialog box, as shown here:

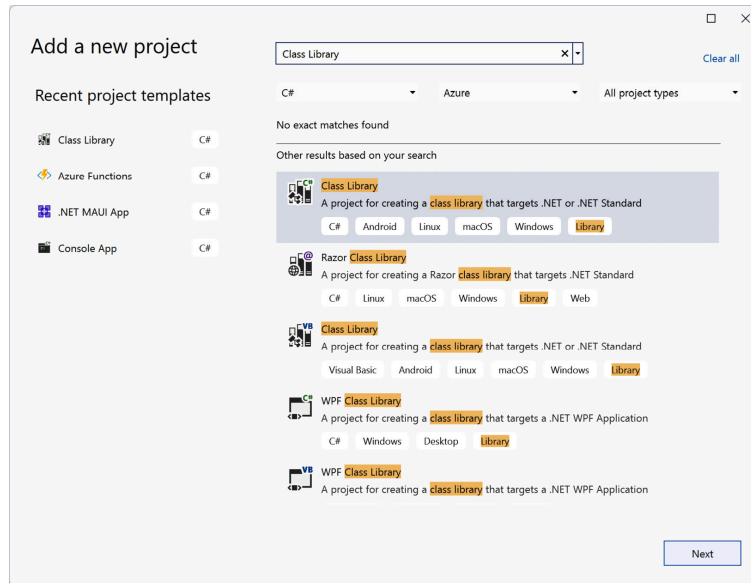


Figure 9.8 – Add a new project

3. Select the **Class Library** template, then click **Next**.
4. In the **Configure your new project** dialog box, enter **StickAndStones.Shared** for the name, as shown in the following screenshot, and click **Next**:

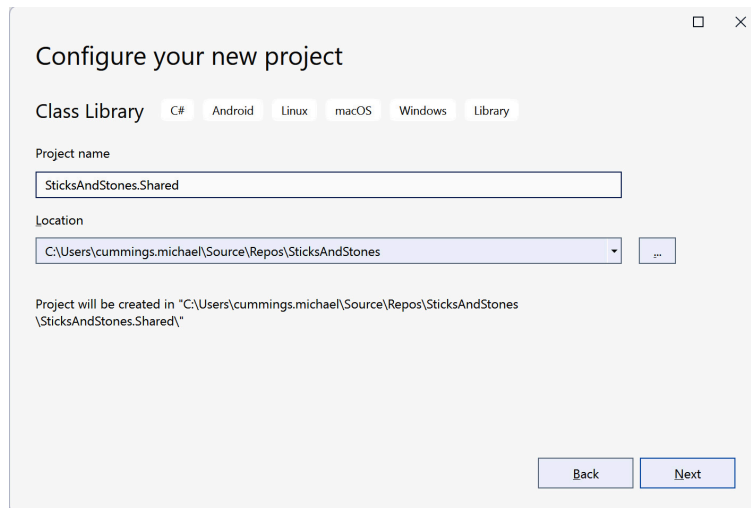


Figure 9.9 – Configure your new project

5. Select the **.NET 6.0 (Long Term Support)** framework project in the **Additional information** dialog, then click **Create**.
6. Delete the **Class1.cs** file that is created as part of the project template.
7. Add a reference to **SticksAndStones.Shared** in the **SticksAndStones.Functions** project.

As we did for the **SticksAndStones.Functions** project, we will change the default namespace by following these steps:

1. Right-click the **SticksAndStones.Functions** project in **Solution Explorer** and select **Properties**.
2. In the **Properties** window, use the search box at the top to search for **Default namespace**.
3. Change the **Default namespace** property value to `$(MSBuildProjectName.Split(".")[0].Replace(" ", "_"))`.
This will split the project name on ., using only the first part and replacing any spaces with underscores.

Now, we can write the code for the function that will return the connection information.

Connecting a player to the game

The first step in the game is to get connected. Being connected adds you to the list of available players so that you or other players can then join a game. As we have done in other projects in this book, first, we will create the models that we need to store or transfer data between the service and the clients. Then, we will implement the **Connect** function itself.

Creating the models

We will need a few functions for the app in *Chapter 10* to call during the life cycle of the app. The first is to establish a connection with the game services, called **Connect**. Essentially, this tells the service that a new or existing player is active and ready for play. The **Connect** function will register the player details and return the connection string to the SignalR hub so that the app can receive messages. We will need a few models before we can complete the function. There needs to be a **Player** model, a **Game** model, and models to assist with passing data between Azure Functions and the SignalR service to the app.

Before we dive into creating the library, we should discuss the naming convention used in this chapter. Having a convention for how you name things will make it easier to determine how the class is used. When the app calls any Azure function, if it needs to send any data, it will do so using a class that has a suffix of **-Request**, and any Azure function that returns data will do so using a class that ends with **-Response**. For any data that is sent via the SignalR hub, we will use a class that has an **EventArgs** suffix. These classes will contain references to our actual models, and just act as a container for the data. Having these classes in place means that you can modify the data being sent or received without affecting the models themselves.

Since this is a two-player game, we need to track a little bit of the state so that we know who is online and what matches are in play. For this project, we will keep the state simple and not involve an actual database, but we will still use Entity Framework to do most of the work for us.

Now that we have created and referenced the new project, and we have a naming convention in place, we can start creating the classes we will need. We will start with the two models, **Player** and **Match**. **Player** repre-

sents each person, while **Game** is a match between two **Player** instances and the state of play. To create the two models, follow these steps:

1. Create a new folder in the **SticksAndStones.Shared** project called **Models**.
2. Create a new class in the **Models** folder called **Player**.
3. Create a **public** property in the **Player** class called **Id** as **Guid** and initialize it to **Guid.Empty**.
4. Create another **public** property called **GamerTag** as **string** and initialize it to **string.Empty**.
5. Create a **public** property named **GameId** as **Guid** and initialize it to **Guid.Empty**.
6. Your **Player** class should now resemble the following code block:

```
namespace SticksAndStones.Models; public class Player { public Guid Id { get; set; } = Guid.E
```

Our model classes will use the **Id** field as a unique identifier so that we can locate each one individually. It will be used to locate specific players for messaging and relating **Match** instances to **Player** instances. **GamerTag** will be the display name for the player, and **EmailAddress** is how we can correlate players if they leave the app and then log back in again. Finally, the **MatchId** property will track whether the player is actively in a game.

Now that we have defined the **Player** class, it's time to define the **Match** class:

1. Create a new class in the **Models** folder called **Match**.
2. Create a **public** property in the **Game** class called **Id** as **Guid** and initialize it to **Guid.Empty**.
3. Create a **public** property called **PlayerOneId** as **Guid**.
4. Add a **public** property named **PlayerOneScore** as **int**.
5. Create another **public** property called **PlayerTwoId** as **Guid**.
6. Add a **public** property named **PlayerTwoScore** as **int**.
7. Create a **public** property called **NextPlayerId** as **Guid**.
8. Create a **public** property called **Sticks** as **List<int>** and initialize it to **new List<int>(24)**.
9. Create a **public** property called **Stones** as **List<int>** and initialize it to **new List<int>(9)**.
10. Create a **public** property called **Scores** as **List<int>** and initialize it to **new List<int>(2)**.
11. Create a **public** property called **Completed** as **bool** and initialize it to **false**.
12. Create a **public** property called **WinnerId** as **Guid** and initialize it to **Guid.Empty**.
13. Add a **public static** method named **New** that takes two parameters, both of the **Guid** type, called **challengerId** and **opponentId**. The method returns an object of the **Game** type. The method should return a new instance of **Game** and assign the **Id** property to **Guid.NewGuid()**, **PlayerOneId** and **NextPlayerId** to **opponentId**, and **PlayerTwoId** to **challengerId**.
14. The **Player** class should now resemble the following code block:

```
using System;using System.Collections.Generic;namespace SticksAndStones.Models;public class Mat
```

The **Player** and **Match** classes will be used for data storage and data transfer between the client and server. Before we go any further with creating our models, let's add the database using Entity Framework. Perform the following steps to add a reference to Entity Framework and create the database context so that the **Player** and **Match** classes can be stored in an **InMemory** database:

1. Add a package reference to **Microsoft.EntityFrameworkCore.InMemory** to the **SticksAndStones.Functions** project.
2. Create a new folder called **Repository** in the **SticksAndStones.Functions** project.
3. Create a class named **GameDbContext** in the **Repository** folder.
4. Modify the constructor for the class to set database options:

```
public GameDbContext(DbContextOptions<GameDbContext> options) : base(options) { }
```

5. Add a public **Players** property to store the **Player** objects:

```
public DbSet<Player> Players { get; set; }
```

6. Add a public **Matches** property to store the **Match** objects:

```
public DbSet<Match> Matches { get; set; }
```

7. Add an override to the **OnModelCreating** method:

```
protected override void OnModelCreating(ModelBuilder modelBuilder){}
```

This method is where we specify to Entity Framework how to relate our classes together in a relational database.

8. Start by declaring the identifiers for each class in the **OnModelCreating** method, as shown here:

```
modelBuilder.Entity<Player>().HasKey<Player>(p => p.Id);modelBuilder.Entity<Match>().Ha
```

9. Entity Framework does not handle our **List<int>** properties properly.

It assumes that since it is a list, they are related instances. To change the default behavior in Entity Framework, we can use the following highlighted code:

```
modelBuilder.Entity<Game>().HasKey(g => g.Id);modelBuilder.Entity<Match>().Property(p =
modelBuilder.Entity<Match>().Property(p => p.Stones).HasConversion(toDb => stri
base.OnModelCreating(modelBuilder);
```

What each block does is define a conversion for the property. A conversion has two Lambda expressions – one from the C# object to the database and the other from the database to the C# object. For our **List<int>** properties, we want to convert a C# **List<int>** into a comma-separated string of integers, and then a comma-separated string of integers to **List<int>**.

toDb is an instance of **List<int>**, so to convert that into a comma-separated list of numbers, we can use the **String.Join** function to join each element of the list with **,** between them.

fromDb is a **string** value containing numbers separated by commas. To convert that into **List<int>**, we can use the **String.Split** method to isolate each number, then pass each number into the **Int.Parse** method to convert the number into an **int** value. **Select** will produce **IEnumerable<int>**; we can use the **LINQ** extension, **ToList**, to convert that into **List<int>**. If it doesn't create a list, we can supply a default list of values, just like we did in the **Match** class itself.

To initialize Entity Framework to use an in-memory database, we need to create a **Startup** method. To create the method and initialize the database, follow these steps:

1. Create a new class named **Startup** in the root of the **SticksAndStones.Functions** project.
2. Modify the class file with the following highlighted code:

```
using Microsoft.Azure.Functions.Extensions.DependencyInjection;using Microsoft.EntityFrameworkCore
namespace SticksAndStones;public class Startup : FunctionsStartup {    public override void Con
```

The **Startup** method will now be called when the **SticksAndStones.Functions** project is loaded at runtime. It will then create a factory for creating instances of the **GameDbContext** class we created previously and initialize it with an in-memory database.

That concludes our setup for Entity Framework and our basic models, **Player** and **Game**. There is one final model that we need to send the SignalR connection information to the client. To create this model, follow these steps:

1. Create a new class in the **Models** folder named **ConnectionInfo**.
2. Add a public property **Url** that is a **string** value.
3. Add another public property named **AccessToken** that is also a **string** value.
4. The **ConnectionInfo** class should look like this:

```
namespace SticksAndStones.Models;public class ConnectionInfo {    public string Url { get; set;
```

With the models now created, we can start creating the **Connect** function.

Creating the Connect function

We will start with a function to connect our player to the game, aptly named **Connect**. This function will expect a partially filled **Player** object to be sent in the request body. The function will return a fully populated **Player** object, a list of the currently connected players, and the connection information needed by the client to connect to the SignalR hub. To make the inputs and outputs cleaner, we will wrap them.

To create the input and output classes, proceed as follows:

1. Create a new class in the **Messages** folder called **ConnectMessages**.
2. Modify **ConnectMessages.cs** so that it looks like this:

```
using SticksAndStones.Models; namespace SticksAndStones.Messages; public record struct ConnectReq
```

For all the classes that will be used to transfer data between the client and Azure Functions or the SignalR service, we will use the **record** syntax. Since these classes will not have any real functionality, their sole purpose is to contain our models. By using a **record** struct, we also improve the memory usage of our functions since a new instance would be created in local memory and not global memory, which requires additional handling. The **record** syntax combines the constructor and property declarations into a single line of code, eliminating a lot of boilerplate code that adds no real benefit.

You will notice that we are using the conventions we discussed in the *Creating the models* section. Classes that have a suffix of **Request** or **Response** are used as input and output for any Azure function. For any data that is sent via the SignalR service, the class will use a suffix of **EventArgs**.

When a new client is connecting, a message will be sent to other users via the SignalR service to indicate that they have connected. This message will also be used to notify when players start or end a game. To create such a message, proceed as follows:

1. Create a new class called **PlayerUpdatedEventArgs** in the **Messages** folder of the **SticksAndStones.Shared** project.
2. Modify the class so that it's a **record** struct with a single **Player** parameter, as shown in the following code snippet:

```
using SticksAndStones.Models; namespace SticksAndStones.Messages; public record struct PlayerUpda
```

Now that we have created the structures needed for the **Connect** function, we can start writing the function itself:

1. Create a new folder called **Hubs**. We will put our service class into this folder.
2. Move the **Function1.cs** file into the **Hubs** folder.
3. Respond **Yes** to the next two prompts for moving the file and adjusting the namespace.
4. Rename the **Function1.cs** file **GameHub.cs** and click **Yes** in the rename prompt.
5. Open the **GameHub.cs** file and rename the class **GameHub**, replace the **internal static** access modifiers and replace them with **public**, and derive it from the **ServerlessHub** base class, as highlighted here:

```
public class GameHub : ServerlessHub
{
    [FunctionName("Function1")] public static async Task<IActionResult> Run([HttpTrigge
```

6. Rename the default **Function1** function **Connect**, removing the **static** modifier as well. The method signature should look like the following

highlighted code snippet:

```
[FunctionName("Connect")]public async Task<IActionResult> Connect( [HttpTrigger(AuthorizationLe
```

The **HttpTrigger** attribute indicates that this function is called by using the HTTP protocol and not by some other means, such as a SignalR message or a timer. The function is only called using the HTTP **POST** method, not **GET**.

7. To send a message to all clients connected to the SignalR hub, we will need another SignalR binding. This time, it is an **output binding**. The output binding is decorated with a SignalR attribute with **HubName** and is of the **IAsyncCollector<SignalRMessage>** type, as follows:

```
[HttpTrigger(AuthorizationLevel.Anonymous, "post", Route = null)] HttpRequest req,[SignalRConne
```

Remove the contents of the **Connect** method and proceed as follows to implement the function:

1. Logging is important in Azure Functions as it helps when debugging in production environments. So, let's add a **log** message:

```
log.LogInformation("A new client is requesting connection");
```

2. The client, a .NET MAUI app, will send **ConnectRequest** in the body of the HTTP request as JSON. To get an instance of **ConnectRequest** from the request body, use the following lines of code:

```
var result = await JsonSerializer.DeserializeAsync<ConnectRequest>(req.Body, jsonOptions);var n
```

You will have to add **using System.Text.Json** to the namespace declarations as well. This uses the **System.Text.Json.JsonSerializer** class to read the contents of the request body and create a **ConnectRequest** object from it. It uses **jsonOptions** to properly deserialize the object.

3. Now, we need to define the **jsonOptions** field. Add the following line of code above the **Connect** method:

```
internal class GameHub : ServerlessHub {    private readonly JsonSerializerOptions jsonOptions
    [FunctionName("Connect")]    public async Task<IActionResult> Connect(
```

JsonSerializerDefaults.Web ensures that the JSON is formatted properly so that Azure Functions and the SignalR service will properly serialize and deserialize the objects. Mainly, it will enforce the following:

1. Property names are case-insensitive
2. All property and object names will be formatted as camelCase
3. Numbers can be quoted
4. If we receive bad player data, return **ArgumentException** to the client, as follows:

```
if (newPlayer is null){    var error = new ArgumentException("No player data.", "Player");    1
```

Since the return type of the function is **IActionResult**, we can't simply return our custom objects. Instead, we need to create an object that

derives or implements **IActionResult** and pass in our result. In the case of errors, we will use **BadRequestObjectResult**, which will accept **Exception** as a parameter in the constructor. **BadRequestObjectResult** will set the HTTP status code to **400**, indicating an error. This status code can then be checked by the client to know whether the request succeeded or not before parsing the body of the response.

5. The next couple of steps will require us to query the database, so we need to add the database context factory to the class. Add the **Microsoft.EntityFrameworkCore** namespace declaration:

```
using Microsoft.Azure.WebJobs.Extensions.SignalRService;using Microsoft.EntityFrameworkCore;usi
```

6. Add a **private** field to store the context factory and a constructor with an argument that will be fulfilled by dependency injection, as follows:

```
private readonly JsonSerializerOptions jsonOptions = new JsonSerializerOptions(JsonSerializerDe
private readonly IDbContextFactory <GameDbContext> dbContextFactory;public GameHub(IDbContextFa
[FunctionName("Connect")]public async Task<IActionResult> Connect(
```

7. Add a namespace declaration for **System.Linq** to allow the use of **Linq** queries:

```
using System;using System.Linq;using System.Text.Json;
```

8. Now that we can access the database context and have determined that the input is valid, we need to make sure that the data is unique, so we will look for **GamerTag** in the database to ensure it isn't in use already by another player, as follows:

```
using var context = dbContextFactory.CreateDbContext();log.LogInformation("Checking for GameTag u
```

The first step is to get a new database context from the factory. The **Linq** query is simple in this case – it uses the **Players** list from the database context to compare **GamerTag** value to other **GamerTags** values. However, we want to exclude a result if it matches **EmailAddress** since that would indicate the records are identical, and this user is just signing back in again.

9. Now, query the **Players** dataset for a player with a matching email:

```
log.LogInformation("Locating Player record."); var thisPlayer = (from p in context.Playe
```

10. If there is no **Player** in **Players** that matches, then add **Player** to the dataset:

```
if (thisPlayer is null){ log.LogInformation("Player not found, creating."); thisPlayer =
```

We assign the **Player** object a new **Guid** so that each **Player** has a unique identifier. This could also be done by Entity Framework; however, we will take care of it here. The context is then used to add the **Player** instance so that it is tracked for any changes. After this, **SaveChangesAsync** will commit all changes to the database.

11. The next step in the **Connect** function is to send a message to all the connected players that a new player has joined. We can do that using the **SendAsync** method. The **SendAsync** method takes two parameters –

the method name as a **string** value that the message is intended for, and the message as an **object** value. To ensure we are sending and receiving the right method, we will create a constant value. Create a new class named **Constants** in the root of the **SticksAndStones.Shared** project, then update it so that it looks like this:

```
namespace SticksAndStones;public static class Constants {    public static class Events    {
```

12. Now, we can notify other connected players that a new player has connected. Open the **GameHub** class and, at the end of the **Connect** method, add the following code:

```
log.LogInformation("Notifying connected players of new player.");    await Clients.All.Send
```

This code uses the **SendAsync** method from the **Clients.All** collection in the **ServerlessHub** base class to send a message to all connected clients. We pass **Constants.Events.PlayerUpdated**, which is the **"PlayerUpdated"** string, as the method name. As arguments, we are sending the **Player** instance wrapped in **PlayerUpdatedEventArgs**. We will handle this message in *Chapter 10*.

13. Now, get the set of available players from the database to send back to the client:

```
// Get the set of available players log.LogInformation("Getting the set of available players.")
```

Using **Linq**, we can easily query the **Players** collection and exclude the current player.

14. At this point, we need to get the **SignalR** connection information from the **SignalR** service. This can be accomplished by calling the **NegotiateAsync** method of the **ServerlessHub** base class. Additionally, so that we can send directed messages to individual users, we will set the **UserId** value for the connection to the player ID value. Add the following line of code to configure and retrieve the **SignalR** connection information:

```
var connectionInfo = await NegotiateAsync(new NegotiationOptions() { UserId = thisPlayer.Id.ToS
```

15. Now that we have all the information that we need to return to the client, we can construct the **ConnectResponse** object. We will use the **ConnectionInfo** class and map the **SignalRConnection** properties to it so that we avoid having to reference the **SignalR** service in the shared library:

```
log.LogInformation("Creating response.");var connectResponse = new ConnectResponse(){    Player
```

16. Once **ConnectResponse** has been initialized, we can return it by using **OkObjectResult**, which will use an **HTTP** response code of **200 OK**:

```
log.LogInformation("Sending response.");return new OkObjectResult(connectResponse);
```

To test the function we just wrote, you can use a **PowerShell** command prompt and the following command, after pressing **F5** in **Visual Studio**:

```
Invoke-WebRequest -Headers @{ ContentType = "application/json" } -Uri http://localhost:7024/api/Co
```

The port number used in the **Uri** parameter may be different for your project. You can get the correct port number by opening the **launchSettings.json** file in the **Properties** folder of the **SticksAndStones.Functions** project. The port number is set in the **commandLineArgs** property, as highlighted here:

```
{ "profiles": { "SticksAndStones.Functions": { "commandName": "Project", "commandLin
```

Replace your port number in the **Uri** parameter after **localhost**:

In the **Body** parameter, you can add the JSON that the command is expecting. For the **Connect** function, this would be **ConnectRequest** and would look like this:

```
{ "player": { "gamerTag": "NewPlayer2", "emailAddress": "newplayer2@gmail.com",
```

The full command will look like this:

```
Invoke-WebRequest -Headers @{ ContentType = "application/json" } -Uri http://localhost:7024/api/Co
```

Go ahead and try out various versions of the command to see how the function reacts.

Now that we can connect players to the game server, let's look at what is needed for the lobby.

Refreshing the lobby

In the **Sticks and Stones App**, which you will create in Chapter 10, once a player has connected, they will move to the lobby page. Initially, the lobby will be populated from the list of players sent in the response from the **Connect** function. Additionally, as each player connects, the lobby will be updated through a **SignalR** event.

But we all get impatient and want a way to refresh the list immediately. So, the lobby page has a way to refresh the list; to do so, it will call the **GetAllPlayers** function.

Let's start by creating the messages needed for **GetAllPlayers**.

Creating the messages

GetAllPlayers takes no parameters, so we only need to create the **GetAllPlayersResponse** type. Follow these steps to add **GetAllPlayersResponse**:

1. In the **SticksAndStones.Shared** project, create a new file named **GetAllPlayers Messages.cs** in the **Messages** folder.

2. Modify the contents of the file so that it looks as follows:

```
using SticksAndStones.Models;namespace SticksAndStones.Messages;public record struct GetAllPlay
```

With the messages created, we can move on to the **GetAllPlayers** function.

Getting all the players

GetAllPlayers is called using the **Http GET** method and has an optional parameter that is passed through **QueryString** using the **id** key. The optional parameter is used to exclude a specific **id** from the returned list. This makes it so that the app can send the current player's **id** and not have it returned in the list. To create the **GetAllPlayers** function, follow these steps:

1. In the **GameHub** class, after the **Connect** method, add the following method declaration:

```
[FunctionName("GetAllPlayers")]public IActionResult GetAllPlayers([HttpTrigger(AuthorizationLev
```

Not much is new here other than using **"get"** instead of **"post"** for the **Http** method, and **Route** is set to **"Players/GetAll"**, which would make the URL for the function **http://localhost:7024/api/Players/GetAll**.

2. In the method, we will process the **id** option parameter. To do so, add the following code:

```
// Exclude the playerId if provided Guid playerId = Guid.Empty;if (req.Query.ContainsKey("id"))
```

In this code, we check for the existence of a key named **id**. If it exists, then its value is retrieved and converted into a **Guid** value and assigned to the **playerId** variable.

3. Next, we can query the database for all players, and exclude **player.Id** using the following code:

```
using var context = contextFactory.CreateDbContext();// Get the set of available players log.Lo
```

4. Finally, return **OkObjectResult** with a new **GetAllPlayersResponse** object initialized with the list of **players**:

```
return new OkObjectResult(new GetAllPlayersResponse(players));
```

Now that we can refresh the lobby with a list of all the players, it's time to match them up for a game.

Challenging another player to a game

To test your skills at this game, you'll need an opponent – someone who would also like to test their skills against yours. This section will build the functionality needed in the **SticksAndStones.Function** project to have one player – the challenger – challenge another player – the opponent – to a game. The opponent has the option to accept the challenge or deny it.

We will also handle the case where the opponent does not respond since they might have put their phone down; this is an edge case.

The interactions in this use case can get tricky, so let's review the following diagram to get a better understanding of what we are building:

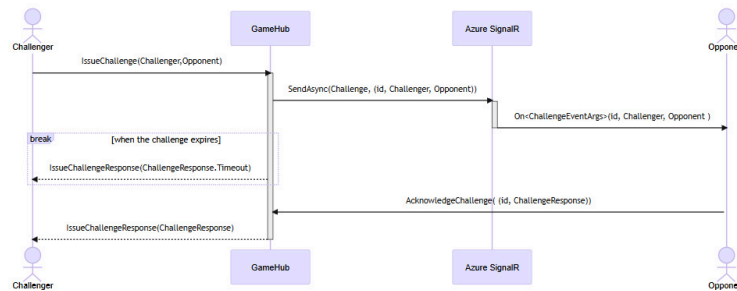


Figure 9.10 – Challenge diagram

The process starts with a user interaction that results in the client making an HTTP request to the **GameHub** instance via the **IssueChallenge** function. The client will pass the challenger and opponent details when making the HTTP call. **IssueChallenge** will create an **Id** value to track this process. **IssueChallenge** will then send a direct message to the opponent via the SignalR hub using the **SendAsync** method. The message will include the **Id** value that was created earlier, the challenger, and the opponent details as an instance of **ChallengeEventArgs**. The opponent's client will receive the message via an **On<ChallengeEventArgs>** event handler. The opponent will then have the choice of accepting or declining the challenge. The response is sent back to the **GameHub** instance using the **AcknowledgeChallenge** function. The **Id** value of the challenge is sent along with **ChallengeResponse**, either **Accept** or **Decline**. A third possibility is **Timeout**. If the opponent never responds, then after a certain amount of time has passed, **Challenge** will **Timeout**. In either event, the result is then returned to the challenger using the response of the **IssueChallenge** call.

Let's get started by defining the messages.

Creating the messages and models

We will start with **IssueChallengeRequest** since that is the first message that is being sent. Follow these steps to create the class:

1. In the **SticksAndStones.Shared** project, create a new file named **ChallengeMessages.cs** under the **Message** folder.
2. Modify the file so that it looks as follows:

```
using SticksAndStones.Models;using System;namespace SticksAndStones.Messages;public record struct
```

As we have in the **Connect** function, we use a **record** struct to eliminate a lot of the boilerplate code needed to define a **struct** value. Our message only needs two **Player** objects – **Challenger** and **Opponent**. The client will have both available when it makes the call to this function. **Challenger**

will be the client making the call to **IssueChallenge** and **Opponent** will be the opposing side.

The **IssueChallenge** function will return **IssueChallengeResponse**. This **IssueChallengeResponse** will have just one field, **Response**, which will be an **enum** value called **ChallengeResponse**. Follow these steps to create **ChallengeResponse**:

1. In the **SticksAndStones.Shared** project, in the **Models** folder, create a new **enum** named **ChallengeResponse**.
2. Add the following values to the **enum** value:
 1. **None**
 2. **Accepted**
 3. **Declined**
 4. **Timeout**

Your code should look like this:

```
namespace SticksAndStones.Models; public enum ChallengeResponse { None, Accepted, Declined, Timeout }
```

To create the remaining messages for the **IssueChallenge** and **AcknowledgeChallenge** functions, follow these steps:

1. Open the **ChallengeMessages.cs** file and add the following declaration at the end of the file:

```
public record struct IssueChallengeResponse(ChallengeResponse Response);
```

2. When the opponent responds to a challenge, they will call the **AcknowledgeChallenge** function and pass an **AcknowledgeChallengeRequest** object as an argument. In the **ChallengeMessages.cs** file, add the following declaration to create **AcknowledgeChallengeRequest**:

```
public record struct AcknowledgeChallengeRequest(Guid Id, ChallengeResponse Response);
```

That completes the messages that are sent or received from the **GameHub** functions for a challenge. That just leaves **ChallengeEventArgs**, which is sent from **GameHub** to the opponent. To create the **ChallengeEventArgs** class, follow these steps:

1. In the **Messages** folder of the **SticksAndStones.Shared** project, create a new file named **ChallengeEventArgs.cs**.
2. Replace the contents of the **ChallengeEventArgs.cs** file with the following:

```
using SticksAndStones.Models; using System; namespace SticksAndStones.Messages; public record struct ChallengeEventArgs(ChallengeResponse Response);
```

3. To add the method name constant for the **SendAsync** method, open the **Constants.cs** file in the **SticksAndStones.Shared** project and add the following highlighted field to the **Events** class:

```
public static class Events {    public static readonly string PlayerUpdated = nameof(PlayerUpda
```

As with the previous message definitions, **ChallengeEventArgs** is also defined as **public record struct**. The parameters are an **Id** value of the **Guid** type and two **Player** objects – one for **Challenger** or the initiator, and one for **Opponent** or the receiver. **Id** will be created in the **IssueChallenge** function and is used to correlate the challenge with the response. This is needed because we are tracking the challenge and if a certain amount of time has passed, we expire the challenge. **Id** is used to track that state and check whether the challenge is still valid if the client responds.

What was not included in the diagram in *Figure 9.10* is a structure that is used to track the challenge in the **GameHub** class. It is only needed while a challenge is active and hasn't timed out. To create the **Challenge** class, follow these steps:

1. Create a file named **Challenge.cs** in the **Models** folder of the **SticksAndStones.Shared** project.
2. Replace the contents of the **Challenge.cs** file with the following code:

```
using System;namespace SticksAndStones.Models;public record struct Challenge(Guid Id, Player Ch
```

As with previous models, we use a **record struct**. The **Challenge** class has various properties – **Id**, **Challenger** as a **Player** type, **Opponent** as a **Player** type, and **ChallengeResponse**, which is called **Response**.

If a player accepts the challenge, then the two players will start a match with each other. Each player will be notified that the match has begun by receiving a **MatchStarted** SignalR event. To create the event and its arguments, follow these steps:

1. Open the **Constants.cs** file in the **SticksAndStones.Shared** project and add the following highlighted field to the **Events** class:

```
public static class Events {    public static readonly string PlayerUpdated = nameof(PlayerUpda
    public static readonly string GameStarted = nameof(MatchStarted);}
```

2. In the **Messages** folder of the **SticksAndStones.Shared** project, create a new file named **MatchStartedEventArgs.cs**.
3. Replace the contents of the **MatchStartedEventArgs.cs** file with the following:

```
using SticksAndStones.Models;namespace SticksAndStones.Messages;public record struct MatchStart
```

That concludes the new messages and models that are needed to allow one player to challenge another to a game of **SticksAndStones**. Next, we will create the first of two functions that will handle the process in the **GameHub** class.

Creating the IssueChallenge function

We'll start with the **IssueChallenge** function. This function is called from the **Challenger** client to start the challenge process. The client will pass their **Player** object, **Challenger**, and the player they are challenging – that is, **Opponent**. These two models are contained in an **IssueChallengeRequest** object. The **IssueChallenge** function will need to perform the following actions:

- Validate input
- Create a challenge object
- Send a challenge to the opponent
- Wait for a response from the opponent
- Send the response to the challenger

To create the function and implement these actions, follow these steps:

1. Open the **GameHub.cs** file in the **SticksAndStones.Functions** project.
2. Add the **IssueChallenge** function declaration, as follows:

```
[FunctionName("IssueChallenge")]public async Task<IssueChallengeResponse> IssueChallenge( [H
```

The **FunctionName** attribute tells Azure Functions that this is an available function. The **req** parameter is an **HttpRequest** object, and the Azure Functions runtime will provide its instance when the function is called. It is attributed by the **HttpTrigger** attribute, which makes the function available via an **Http** API call. **HttpRequest** must use the **POST** method and not **GET** when making the call and the function's **Route** or **Url** will end with **Challenge/Issue**. The function returns **IssueChallengeResponse** to the caller.

3. The first action the function performs is to validate the inputs. The **IssueChallengeRequest** object is sent as part of the body of the **Http** POST request. To retrieve the instance, use the following code in the **IssueChallenge** function:

```
{    var result = await JsonSerializer.DeserializeAsync<IssueChallengeRequest>(req.Body, jsonOp
```

This is the same way we retrieved the arguments that were passed in the **Connect** function. The main difference is that the type of object being returned by the **DeserializeAsync** method is **IssueChallengeRequest**, not **ConnectRequest**. The **jsonOptions** field is already defined in the **GameHub** class.

4. Now, we need to check whether the challenger and opponent are valid. Valid means that they exist in our database, and neither are currently in a match. We will use **Linq** queries to check our database for each object. Add the following code to the **IssueChallenge** function to verify that the players exist:

```
using var context = contextFactory.CreateDbContext();Guid challengerId = result.Challenger.Id;v  
  
Guid opponentId = result.Opponent.Id;var opponent = (from p in context.Players
```


First, we capture the **Id** value of the player. We use this **Id** to query the **Players DbSet** in the database context for a matching **Id** and return the **Player** object if it exists; otherwise, we return **null**. If the object is **null**, then we exit the function by throwing **ArgumentException** and passing the name of the object as the **paramName** argument and a message detailing the issue. This can be used on the client to display an error message.

5. The following code will check whether the players are currently engaged in a match with another player. Add the following code to the end of the **IssueChallenge** function:

```
var challengerInMatch = (from g in context.Matches
                        where g.PlayerOneId == challengerId || g.PlayerTwoId == challengerId
                        select g).Any();
var opponentInMatch = (from g in context.Matches
                       where g.PlayerOneId == opponentId || g.PlayerTwoId == opponentId
                       select g).Any();
if (challengerInMatch)
    throw new ArgumentException(paramName: nameof(challenger), message: $"{challenger.GamerTag}");
if (opponentInMatch)
    throw new ArgumentException(paramName: nameof(opponent), message: $"{opponent.GamerTag} is
```

Again, we use **Linq** to query the database, but this time, we are querying the **Matches DbSet**. We are not looking for the **Match** instance, just the fact that one does exist, where either **PlayerOneId** or **PlayerTwoId** is the player's **Id** value. We use the **Any** function to return **true** if there are any results and **false** if there are no results. Again, we throw **ArgumentException** if either player is in a match with an appropriate message.

6. At this point, we have validated that both players exist and can join a new game. We will need to capture the game **Id** value if the challenge is accepted, so let's create the variable and log some details before moving on:

```
Guid matchId = Guid.Empty; log.LogInformation($"{challenger.GamerTag} has challenged {opponent.G
```

The next step in the **IssueChallenge** function is to create a **Challenge** object. But because we want to track how long **Challenge** is waiting so that we can time it out, we need a helper class to abstract that detail away from the function.

DON'T REINVENT THE WHEEL

*The implementation of **ChallengeHandler** is heavily based on **AckHandler** from the **Azure SignalR AckableChatRoom** sample. The source for the sample is available at <https://github.com/aspnet/AzureSignalR-samples/tree/main/samples/AckableChatRoom>.*

Let's create the **ChallengeHandler** class by following these steps:

1. Create a new folder in the **SticksAndStones.Functions** folder named **Handlers**.

2. Create a new class named **ChallengeHandler** in the **Handlers** folder, and change the access modifier from **internal** to **public**.
3. Add a constructor for the class that has three parameters – **completeAcksOnTimeout** as **bool**, **ackThreshold** as **TimeSpan**, and **ackInterval** as **TimeSpan**. The constructor will create a timer to periodically clear out old challenges and store the **ackThreshold** value in a class field. The class's contents should look like this:

```
private readonly TimeSpan ackThreshold; private readonly Timer timer;
public ChallengeHandler(bool completeAcksOnTimeout, TimeSpan ackThreshold, TimeSpan ackInterval
```

You will need to add a **using** declaration for the **System.Threading** namespace to use the **Timer** type. The **CheckAcks** method will be created later in this section.

4. To provide some reasonable defaults for the constructor, we will create a parameterless constructor and provide the defaults, as shown in the following snippet:

```
private readonly Timer timer;
public ChallengeHandler() : this(    completeAcksOnTimeout: true,    ackThreshold: TimeSpan.From
public ChallengeHandler(bool completeAcksOnTimeout, TimeSpan ackThreshold, TimeSpan ackInterval
```

This will provide the default values to the main constructor.

5. Now, to create a new **Challenge**, the **IssueChallenge** function will call a method named **CreateChallenge**, as shown here:

```
public (Guid id, Task<Challenge> responseTask) CreateChallenge(Player challenger, Player oppone
```

Add this method to the **ChallengeHandler** class, below the main constructor. The method will use **TaskCompletionSource** to track the **Challenge** object. The **Challenge** and **TaskCompletionSource** instances are held by **ChallengeRecord**, which looks like this:

```
private record struct ChallengeRecord(Guid Id, TaskCompletionSource<Challenge> ResponseTask, Da
```

Add this declaration to the top of the **ChallengeHandler** class. This **record** has an **Id** value – for uniqueness, **TaskCompletionSource** – a **DateTime** value to track the creation time, and the **Challenge** object itself. We keep a list of **ChallengeRecord** instances in another field called **handlers**. The **handlers** field, which is declared right after the **ChallengeRecord** class, is as follows:

```
private readonly ConcurrentDictionary<Guid, ChallengeRecord> handlers = new();
```

We use **ConcurrentDictionary** since we may be accessing the field from several different threads at the same time.

ConncurrentDictionary is designed to prevent data corruption in multithreaded situations, like this one.

Once **TaskCompletionSource**, **Challenge**, and **ChallengeRecord** have been created, the **Challenge Id** value and the **Task** value associated with **TaskCompletionSource** are returned as a **Tuple** value to the **IssueChallenge** function. We will see what happens to that data later in this section when we complete the **IssueChallege** function.

Finally, to resolve the missing namespaces, add the following highlighted namespace declarations to your **ChallengeHandler** class file:

```
using SticksAndStones.Models;
using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
```

6. To acknowledge the challenge, either accepting it or declining it, the **AcknowledgeChallenge** function will call a method named **Respond**. The **Respond** method will remove **ChallengeRecord** from the dictionary, if it exists, and return the associate **Challenge**. If there is no **ChallengeRecord**, then a new empty **Challenge** record is returned, as shown in the following code:

```
public Challenge Respond(Guid id, ChallengeResponse response){ if (handlers.TryRemove(id, ou
```

7. The **CheckAcks** method, which is called periodically to check for **Challenge** objects that have expired and have not been responded to, looks like this:

```
private void CheckAcks(){ foreach (var pair in handlers) { var elapsed = DateTime.
```

This method will iterate over all the pairs in the **handlers** dictionary. For each one, it will determine whether the elapsed time is greater than the threshold provided in the constructor. If it is, then the task fails with a **TimeoutException** error.

8. To wrap up this class, we need to make sure that we clean up any remaining tasks when the service shuts down. We will handle canceling tasks in a **Dispose** method, which is implemented via the **IDisposable** interface. Add the **IDisposable** interface to the **ChallengeHandler** class and add the following **Dispose** method to the end of the class:

```
public void Dispose(){ timer?.Dispose(); foreach (var pair in handlers) { pair.
```

The **Dispose** method will dispose of the timer since we don't want that firing any longer. Then, it iterates over the handlers and cancels each of the tasks.

That should complete the **ChallengeHandler** class. We can now resume the implementation of the **IssueChallenge** function:

1. Open the **GameHub.cs** file and locate the constructor, modifying it as highlighted in the following code:

```
private readonly GameDbContext context;private readonly ChallengeHandler challengeHandler;
public GameHub(GameDbContext dbContext, ChallengeHandler handler){ context = dbContext; c
}
```

Since we will need the **ChallengeHandler** class, and it needs to maintain state, we will use dependency injection and have the Azure Functions runtime supply us with the instance.

2. Open the **Startup.cs** file in the **SticksAndStones.Function** project and add the following line of code at the end of the **Configure** method:

```
builder.Services.AddSingleton<ChallengeHandler>();
```

This will register **ChallengeHandler** with dependency injection to allow the Azure Functions runtime to manage the instance creation and lifetime.

3. Open the **GameHub.cs** file and navigate to the bottom of the **IssueChallenge** function.
4. Add the following lines of code:

```
var challengeInfo = challengeHandler.CreateChallenge(challenger, opponent);log.LogInformation($
log.LogInformation($"Waiting on response from {opponent.GamerTag} for challenge[{challengeInfo.
```

This code will first call **CreateChallenge** using the **ChallengeHandler** instance that we are getting in the constructor. **challengeInfo** is a **Tuple** value of the **Challenge Id** type and **task**.

Next, the opponent is sent a SignalR **Challenge** message with **ChallengeEventArgs**. This message is sent slightly differently since this message will only be sent to the client that matches the opponent's **Id**.

5. Now, we need to wait for the opponent's response, or a timeout from **ChallengeHandler**, by using the following code:

```
ChallengeResponse response;try { var challenge = await challengeInfo.responseTask.ConfigureA
```

The real trick in this code is the **challengeInfo.responseTask** await. **responseTask** is the task that is created as part of **TaskCompletionSource** in **ChallengeHandler**. By awaiting it, we do not continue until either the **Respond** method is called and the task is completed, or the task is failed by setting a **TimeoutException** error in the **CheckAcks** method of **ChallengeHandler**. Once one of those conditions is **true**, the method completes and we can get the response from the returned **Challenge**, or in the case of a timeout, handle the exception and return the response to the client in a new instance of **IssueChallengeResponse**.

The **IssueChallenge** function is now complete. The client can call the function and it will send a message to the opponent's client and wait for the response. If the opponent client does not respond in a defined amount of time, which is 30 seconds by default, then the challenge will time out. Now, let's work on accepting or declining a challenge. As with the **Connect** function, you can try it out using the command line. You just need to connect two players, and then have one challenge the other!

Creating the AcknowledgeChallenge function

The **AcknowledgeChallenge** function is used by the client to respond to an open challenge from another player. Let's create the function by following these steps:

1. Add a new function to the **GameHub** class, as follows:

```
[FunctionName("AcknowledgeChallenge")]public async Task AcknowledgeChallenge( [HttpTrigger(A
{}
```

2. In the body of the function, deserialize the arguments using the following line of code:

```
var result = await JsonSerializer.DeserializeAsync<AcknowledgeChallengeRequest>(req.Body, json0
```

3. Use **challengeHandler** to **Respond** to the challenge:

```
var challenge = challengeHandler.Respond(result.Id, result.Response);if (challenge.Id == Guid.E
```

4. If the response is **Declined**, then just log a message:

```
var challenger = challenge.Challenger;var opponent = challenge.Opponent;if (result.Response ==
```

5. If the response is **Accepted**, then create a match and notify the players:

```
if (result.Response == ChallengeResponse.Accepted)
{
    log.LogInformation($"{opponent.GamerTag} has accepted the challenge from {challenger.GamerT
using var context = contextFactory.CreateDbContext();
var game = Match.New(challenger.Id, opponent.Id);
context.Matches.Add(game);
opponent.MatchId = challenger.MatchId = match.Id;
context.Players.Update(opponent);
context.Players.Update(challenger);
context.SaveChanges();
log.LogInformation($"Created match {match.Id} between {opponent.GamerTag} and {challenger.G
// Create Group for Game
await UserGroups.AddToGroupAsync(opponent.Id.ToString(), $"Match[{match.Id}]");
await UserGroups.AddToGroupAsync(challenger.Id.ToString(), $"Match[{match.Id}]");
await Clients.Group($"Match[{match.Id}]").SendAsync(Constants.Events.MatchStarted, new Matc
await Clients.All.SendAsync(Constants.Events.PlayerUpdated, new PlayerUpdatedEventArgs(oppo
await Clients.All.SendAsync(Constants.Events.PlayerUpdated, new PlayerUpdatedEventArgs(chal
}
```

So, ignoring all the logging, since that is non-functional, the preceding code starts by creating a new **Match** instance and assigning **PlayerOneId**, **PlayerTwoId**, and **NextPlayerId**. The **Match** object's **Id** property is then assigned to both of the players, and all the changes are saved to the database.

Next, is the SignalR messages. First, we create a SignalR group with just the two players in it and use the **Match** object's **Id** property in the name. This way we can send messages to the group and both players will receive them.

The first message we will send will indicate the start of a new game and it will send the **match** instance wrapped in **MatchStartedEventArgs**.

Finally, we send a message for each player to all players, indicating a change in their status.

That completes the functionality for one player to challenge another player to a match of Sticks and Stones! It's time to move on to playing a match. But first, we will need a function to return the game to the player.

Getting the match

You may be wondering why we need this functionality since, in the previous function, we sent the **Match** object to both players through a SignalR message. The answer is rather simple – if the user accidentally closes the Sticks and Stones app while in the middle of a game, then when they return to the Sticks and Stones app and log back in, the app will detect that they are still in a match and navigate to the **Match** page. It will use this function to retrieve the **Match** object in this case since it wasn't sent during **Connect**, just **Id**.

So, let's create a function to return a game by its **Id**, starting with the messages.

Creating the messages

This function will only need a response message object. Unlike the previous functions, the **GetMatch** function will use the **Http** GET method, and we will pass the match **Id** value as part of the URL. The response from the **GetGame** function will be the **Match** instance. To create the **GetGameResponse** message, follow these steps:

1. In the **SticksAndStones.Shared** project, create a new file named **GetGameMessages.cs** in the **Messages** folder.
2. Modify the contents of the file so that it's as follows:

```
using SticksAndStones.Models; namespace SticksAndStones.Messages; public record struct GetMatchRe
```

With the response message class in place, we can create the **GetMatch** function.

Getting a match by its ID

The **GetMatch** function will accept a single integer named **id** as a parameter. The parameter is bound to a part of the URL that's used to call the function. Let's look at an example. If we wanted to get **Match** identified by a **Guid** type of **c39c7490-f4bc-425a-84ab-0a4ad916ea48**, then the URL would be **http://localhost:7024/api/Game/c39c7490-f4bc-425a-84ab-0a4ad916ea48**.

Follow these steps to implement the **GetMatch** function:

1. Open the **GameHub.cs** file in the **Hubs** folder of the **SticksAndStones.Functions** project.
2. Add the following method declaration after the **AcknowledgeChallenge** method:

```
[FunctionName("GetMatch")]
public IActionResult GetMatch(
    [HttpTrigger(AuthorizationLevel.Function, "get", Route =
        "Match/{id}")] HttpRequest req,
    Guid id, ILogger log){}
```

There are a few differences from the other functions. First, the **Http** method that's used is **"get"**, not **"post"**. Second, **Route** is set to

"Game/{id}"; {id} in **Route** tells the Azure Functions runtime that this function has a parameter named **id** and that the value in that position of the URL should be passed in as an argument. You can see that the third change is that there is an **id** parameter of the **Guid** type. This means that whatever is on the URL in the {id} position must be able to be converted into the **Guid** type; otherwise, the Azure Functions runtime will return an HTTP **500 Internal Server** error.

3. To query our database for the **Match** object that matches the **id** value, use the following lines of code:

```
using var context = contextFactory.CreateDbContext(); Match match = (from m in context.Matches w
```

4. If the method gets this far, then it has been completed successfully, so we can return **OkObjectResult**. The object that's returned will be a **GetMatchResponse** instance with the **Match** instance that was found, or **null**:

```
return new OkObjectResult(new GetMatchResponse(match));
```

Since this function uses the HTTP GET method, you can test it out in your favorite browser:

1. Press *F5* to start the project in debug mode.
2. Wait for the service to start, then copy the URL for the **GetMatch** function from the output window – for example, `http://localhost:####/api/Game/{id}`, where **####** is your port number.
3. Open your browser and paste the URL in the address bar.
4. Change **{id}** to anything.
5. Press *Enter*.

You should get an error page in your browser. Try a valid **Guid** value such as **c39c7490-f4bc-425a-84ab-0a4ad916ea48**. You should get a response similar to the following:

```
{  "match": null }
```

Since there are no active games, you won't be able to retrieve an actual **Match** instance.

Now that we can retrieve the **Match** object, we will tackle how players make and receive moves and how to determine the score and winner of the game.

Playing the game

Sticks and Stones is an interactive, fast-paced, turn-based game.

Let's review the following diagram to get a better understanding of what we are building:

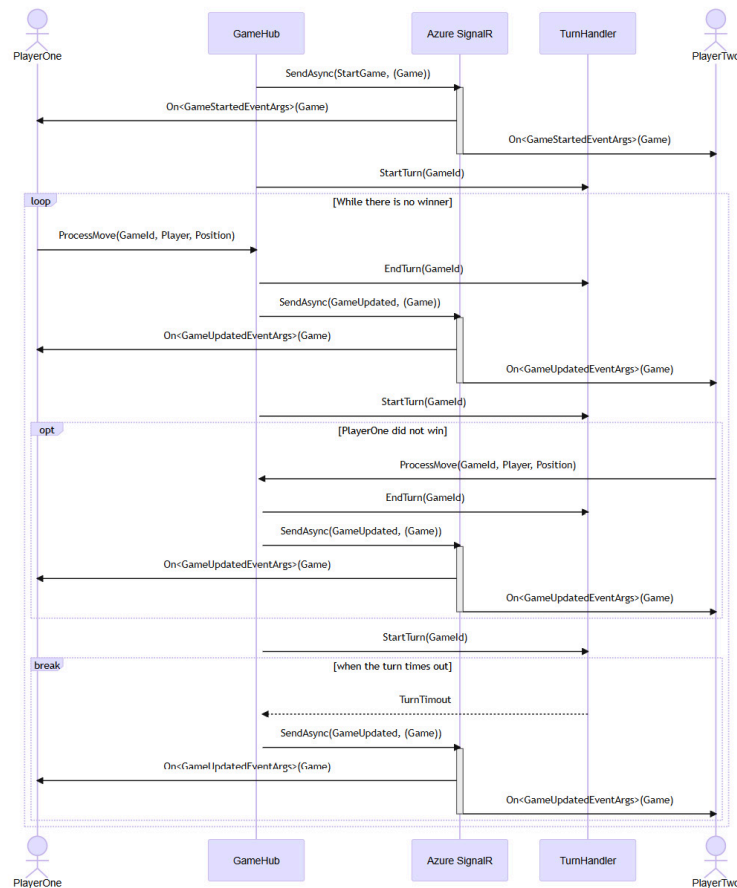


Figure 9.11 – Processing turns

Once a match has started, players will take turns choosing a location to place one of their sticks. The client application will then send a message to the GameHub's **ProcessTurn** function. The **ProcessTurn** function will then validate the move, recalculate the score, check for a winner, and finally, send an update to the players.

Creating the ProcessTurn messages and models

The **ProcessTurn** function has three parameters – the **Match Id**, the player making the move, and the position of the move. The function will return an updated **Match** instance. Follow these steps to add the **ProcessTurn** messages:

1. Add a new file named **ProcessTurnMessages.cs** to the **Messages** folder in the **SticksAndStones.Shared** project.
2. Modify the contents of the file so that it looks as follows:

```
using SticksAndStones.Models;using System;namespace SticksAndStones.Messages;public record struct ProcessTurnResponse(Match Match);
```

As part of the turn, **ProcessTurn** will send the updated **Match** instance to the players. This will require a new SignalR event. Perform the following steps to add it:

1. **SaveMatchAndSendUpdates** sends a new event via SignalR to the clients, so we need to add that to our constants. Add the highlighted

code in the following snippet to the **Constants.cs** file in the **SticksAndStones.Shared** project:

```
public static class Events    {        public static readonly string PlayerUpdated = nameof(PlayerUpdated);
    public static readonly string MatchUpdated = nameof(MatchUpdated);
    }
```

2. Add the **MatchUpdatedEventArgs** class that we used to send the updated **Match** to the players when the **MatchUpdated** event is set by adding a new file named **MatchUpdatedEventArgs.cs** to the **Messages** folder in the **SticksAndStones.Shared** project.
3. Modify the contents of the file so that it's as follows:

```
using SticksAndStones.Models; namespace SticksAndStones.Messages; public record struct MatchUpdat
```

That concludes the additional models, events, and messages that are used by the **ProcessTurn** function. Next, we can start working on the **ProcessTurn** (P-Code) function.

Processing turns

The **ProcessTurn** function has a few responsibilities. It will need to do the following:

- Validate the turn
- Make the necessary changes to the **Match** object
- Recalculate the score
- Determine whether there is a winner
- Notify the players

To start the implementation of the **ProcessTurn** function, we will create stubs for each of the methods that we will call when processing a turn. Follow these steps to create the method stubs:

1. Add a new method declaration to the **GameHub** class for the **ProcessTurn** function:

```
[FunctionName("ProcessTurn")] public async Task<IActionResult> ProcessTurn(
    [HttpTrigger(AuthorizationLevel.Function, "post", Route = $"Game/Move")] HttpRequest req,
    ILogger log)
{ }
```

2. Add a new method declaration for **ValidateProcessTurnRequest**:

```
private Exception ValidateProcessTurnRequest(ProcessTurnRequest args)
{     return null; }
```

The method accepts **ProcessTurnRequest** as the only argument. If there are no errors in the arguments, then it will default to returning **null**. If there is an error, then it will return an **Exception** error.

3. We will use another method to verify the match state before processing the move. Create a new method in the **GameHub** class named **VerifyMatchState**, as follows:

```
private Exception VerifyMatchState(Match match, ProcessTurnRequest args)
{
    return null;
}
```

Just like with **ValidateProcessTurnRequest**, we will return **null** for the error if everything is okay; otherwise, we will return an error.

Now that we have the helper method signatures in place, let's implement them, starting with **ValidateProcessTurnRequest**. Follow these steps to add the implementation to **ValidateProcessTurnRequest**:

1. In **ValidateProcessTurnRequest**, add the following at the top of the method to check for a valid position:

```
if (args.Position <= 0 || args.Position > 23)
{
    return new IndexOutOfRangeException("Position is out of range, must be between 1 and 24");
}
```

2. In **ValidateProcessTurnRequest**, add the following at the top of the method to check for a valid player:

```
if (args.Player is null){    return new ArgumentException("Invalid Player");}
```

3. In **ValidateProcessTurnRequest**, add the following at the top of the method to check for a valid game **Id** value:

```
if (args.MatchId == Guid.Empty){
    return new ArgumentException("Invalid MatchId");}
```

That completes the **ValidateProcessTurnRequest** method. Now, we can add the code to **VerifyMatchState**, as follows:

1. In **VerifyMatchState**, add the following at the top of the method to check that the position hasn't already been played:

```
if (match.Sticks[args.Position] != 0){    return new ArgumentException($"Position [{args.Position}] is already played");}
```

2. In **VerifyMatchState**, add the following at the top of the method to check that the correct player is taking their turn:

```
if (args.Player.Id != game.NextPlayerId){    return new ArgumentException($"It is not {args.Player.Name}'s turn");}
```

3. In **VerifyMatchState**, add the following at the top of the method to check that the game isn't over already:

```
if (match.WinnerId != Guid.Empty){    return new ArgumentException("Match is complete");}
```

4. In **VerifyMatchState**, add the following at the top of the method to check that the game object exists:

```
if (match is null)
{
    return new ArgumentException("Invalid MatchId");
}
```

Now that we have created these helper methods, we can implement the **ProcessTurn** method by following these steps:

1. Deserialize the arguments that are passed to the **ProcessTurn** function using **JsonSerializer**, as follows:

```
var args = await JsonSerializer.DeserializeAsync<ProcessTurnRequest>(req.Body, jsonOptions);
```

2. In the **ProcessTurn** method, we can call **ValidateProcessTurnRequest**.

If there is an error, we can handle it, as follows:

```
var error = ValidateProcessTurnRequest(args);
if (error is not null)
{
    log.LogError(error, "Error validating turn request");
    return new BadRequestObjectResult(error);
}
```

3. With the arguments verified, we can query the database for the game, and fail if it doesn't exist:

```
using var context = contextFactory.CreateDbContext();
var game = (from g in context.Matches where m.Id == args.MatchId select m).FirstOrDefault() ??
```

4. Now, we can call **VerifyGameState**. If there is an error, we can handle it, as follows:

```
error = VerifyGameState(game, args);
if (error is not null)
{
    log.LogError(error, "Error validating game state");
    return new BadRequestObjectResult(error);
}
```

5. We must do one final check before making the move and updating the scores – we need to check to see whether the player made their selection before their turn expired using the following code:

```
if (turnHandler.EndTurn(args.GameId) == TurnHandler.TurnStatus.Forfeit)
{
    error = new ArgumentException($"The turn has expired.");
    log.LogError(error, $"Player did not respond in the time allotted.");
    return new BadRequestObjectResult(error);
}
```

With the validation of the input and game out of the way, we can now focus on applying the player's move to the current state of the game, updating the score, and determining a winner. To make the code for updating the score simpler, we will need a complex data structure. Let's explore this further.

When a player chooses a location to place one of their sticks, it may complete a square. If it does, then the player who placed the stick gets the square and an additional five points. The trick is how to determine that a stick has completed a square. We are storing the state of all sticks and stones as an array of integers. What we need is a map from a stone index

(0–8) to the sticks that make up its sides. But we can simplify the logic a bit more once we know what sticks make up a square since we are only interested in a single stick, and a single stick can complete, at most, two squares. So, we can now have a structure that maps each stick position (0–23) to an array of tuples. Each tuple has an integer that is the index for the stone and another integer array that is the other three stick indexes that make up the square.

Let’s use an example to illustrate this. Pretend that we have a game that’s in the following state:

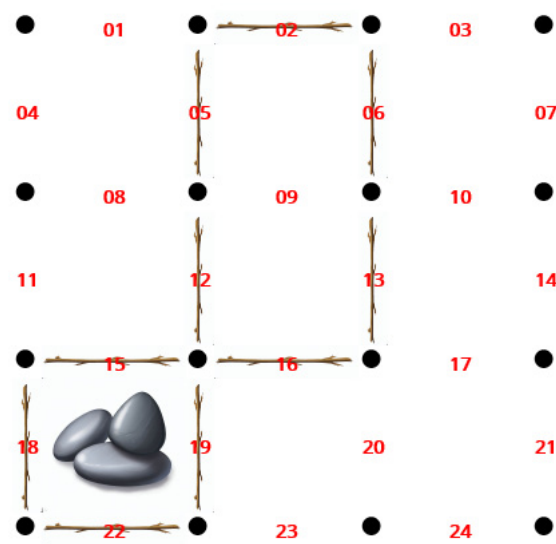


Figure 9.12 – Sample view of the board game

Now, pretend that a player has chosen to place their stick at location 9, highlighted in aqua:

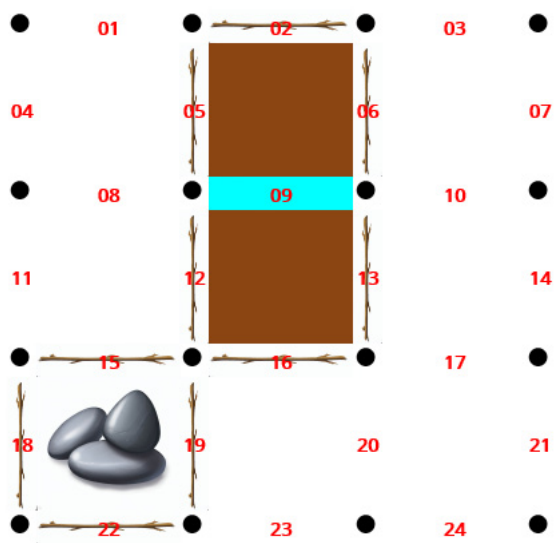


Figure 9.13 – Board game with stick placement highlighted

We will only need to check the two squares highlighted in brown. This means that we need to check whether there are sticks in positions 2, 5, and 6 for the stone in the upper brown box, and 12, 13, and 16 for the stone in the lower brown box.

This means we need two tuples – one for stone 2 and a second for stone 5 – each with an array of integers of the sides – for example, { (2, {2, 5, 6}), (5, {12, 13, 16}) }. Using that data, we can check the two possible squares that could be completed by placing a stick at position 9.

Using old-fashioned sticky notes with a pencil and eraser, we can determine that the complete mapping will look like this:

```
(int stone, int[] sticks)[][] stickToStoneMap = new (int, int[])[][] { /* 1 */ new (int, int[])[] {
```

Add the preceding code to the **GameHub** class after the **turnHandler** field declaration. Now that we have declared the data structure for finding completed boxes, let's continue processing the turn:

1. Return the **ProcessTurn** method, and add the following code at the end:

```
match.Sticks[args.Position] = args.Player.Id == match.PlayerOneId ? 1 : -1;
```

This will assign the position **-1** or **1**, depending on who the active player is. We will use a value of **-1** and **1** later in determining a winner, in the case of three stones in a row.

2. Now that we have placed the stick, we need to adjust the player's score, as follows:

```
if (args.Player.Id == game.PlayerOneId){ match.PlayerOneScore += 1;}else { match.PlayerTw
```

3. The following code will use the data structure from earlier to determine whether placing the stick completed any squares:

```
// Determine if this play creates a square
foreach (var tuple in stickToStoneMap[args.Position])
{
    if (tuple.stone == 0) continue;
    var stickCompletesABox =
    (
        Math.Abs(match.Sticks[tuple.sticks[0] - 1]) +
        Math.Abs(match.Sticks[tuple.sticks[1] - 1]) +
        Math.Abs(match.Sticks[tuple.sticks[2] - 1])
    ) == 3;
    if (stickCompletesABox)
    {
        // If so, place stone, and adjust score
        var player = args.Player.Id == match.PlayerOneId ? 1 : -1;
        match.Stones[tuple.stone - 1] = player;
        if (player > 0)
        {
            match.PlayerOneScore += 5;
        }
        else
        {
            match.PlayerTwoScore += 5;
        }
    }
}
```

This code will iterate over the tuples declared at the array position of the newly placed stick, adjusting for C# arrays being 0-based. It will then use the array of stick positions from the tuple to index into the array of sticks in the match. The value at that location will be either **1** for player one, **-1** for player two, or **0** for unclaimed. We can add the absolute value of all three locations and if it is 3, then the newly placed stick completes a box. If so, then assign the stone location from the tuple, adjusting for 0-based arrays again, to the player, and give them five points.

4. To help determine whether the match is over, we are going to use a couple of helper functions to make the code cleaner and easier to read. The first of those returns a **boolean** value if all the sticks have been played in the match. Add the following code to the end of the **GameHub** class:

```
private static bool AllSticksHaveBeenPlayed(Match match)
{
    return !(from s in match.Sticks where s == 0 select s).Any();
}
```

This function uses a straightforward LINQ query to search the **Sticks** array for any element that has a value of **0**, meaning unclaimed. If there are, the function returns.

5. The next function is a little more complex as it is used to determine whether a player has three stones in a row, either horizontally, vertically, or diagonally, and returns the **Id** value of the player that does. Add the following code to the end of the **GameHub** class:

```
private static int HasThreeInARow(List<int> stones){    for (var rc = 0; rc < 3; rc++)    {
```

This method starts by checking all the rows and columns for 3 stones in a row, for the same player. Since there are nine stones arranged in a 3x3 grid, we only need to check three columns and three rows. Using a single iterator, each row or column is checked by adding the values stored at each position in the row or column and if the absolute value of the sum is 3, then a single player has a winning row. If the sum is positive, player one has won; otherwise, player two has won. Since there are only two possible diagonals, those checks use the same logic but are done individually, rather than looping.

6. Now, we can use those two functions to determine whether there is a winner. To do so, we can use the following code at the end of the **ProcessTurn** function:

```
// Does one player have 3 stones in a row?
var winner = Guid.Empty;
var threeInARow = HasThreeInARow(match.Stones);
if (threeInARow != 0)
    winner = threeInARow > 0 ? match.PlayerOneId : match.PlayerTwoId;
if (winner == Guid.Empty) // No Winner yet
{
    // Have all sticks been played, if yes, use top score.
    if (HaveAllSticksBeenPlayed(match))
    {
        winner = match.PlayerOneScore > match.PlayerTwoScore ? match.PlayerOneId : match.Player
```

```
    }
}
```

Here, we use the two methods we just created to do the main checks and assign the winner. We capture the winner as the **Guid** type from the **Id** property of the **Player** class, so some translation is needed.

7. Next, we can set the next player's turn, or if there is a winner, complete the match, as follows:

```
if (winner == Guid.Empty)
{
    match.NextPlayerId = args.Player.Id == match.PlayerOneId ? match.PlayerTwoId : match.PlayerOneId;
}
else
{
    match.NextPlayerId = Guid.Empty;
    match.WinnerId = winner;
    match.Completed = true;
}
```

8. The final steps are to save any changes we have made and send updates to the players. We will use a helper method called **SaveGameAndSendUpdates** to handle that as we will need the same code when a turn expires. Add the following code to the end of the **GameHub** class:

```
private async Task SaveMatchAndSendUpdates(GameDbContext context, Match match)
{
    context.Matches.Update(match);
    await context.SaveChangesAsync();
    await Clients.Group($"Match[{match.Id}]").SendAsync(Constants.Events.MatchUpdated, new MatchUpdate { MatchId = match.Id });
    if (match.Completed)
    {
        await UserGroups.RemoveFromGroupAsync(match.PlayerOneId.ToString(), $"Match[{match.Id}]");
        await UserGroups.RemoveFromGroupAsync(match.PlayerTwoId.ToString(), $"Match[{match.Id}]");
    }
}
```

This function will save the current match state to the database, then sends a message to the SignalR group for the match indicating that there have been updates to the match. If the match is over, then we remove the players from the group.

1. The following final three lines of code complete the **ProcessTurn** method:

```
await SaveMatchAndSendUpdates(context, match);
return new OkObjectResult(new ProcessTurnResponse(match));
```

After saving the match changes and notifying the players of the match updates, if the match is not over yet, we notify the next player that it is their turn to play. To wrap things up we return the updated match object back to the player that just made their move.

2. We also need to call **SaveGameAndSendUpdates** when there is an error after calling **VerifyGameState**. Modify that section of code in **ProcessTurn** using the following snippet:

```
error = VerifyMatchState(game, args);
if (error is not null)
{
    await SaveMatchAndSendUpdates(game);
    log.LogError(error, "Error validating match state.");
    return new BadRequestObjectResult(error);
}
```

We have now completed all the required functions to connect a player to the service, challenge another player to a match, and then process each player's turn and determine the winner.

Let's take a short look back at what we have accomplished so far in this chapter. We started by creating the Azure services that our game server backend would need, a SignalR service for real-time communication, and finally, the Functions service to host our backend functions. We then implemented the Azure functions that would provide the functionality for our game:

- **Connect:** To register players to the game service
- **IssueChallenge:** To allow one player to request a game with another player
- **AcknowledgeChallenge:** To accept or decline a request
- **ProcessTurn:** To manage the gameplay between two players and determine the winner

Our backend is now complete, and we are ready to publish it to Azure so that we can consume the services from the game app in *Chapter 10*.

Deploying the functions to Azure

The final step in this chapter is to deploy the functions to Azure. You can do that as a part of a **continuous integration/continuous deployment (CI/CD)** pipeline – for example, with Azure DevOps. But the easiest way to deploy the functions, in this case, is to do it directly from Visual Studio. Perform the following steps to deploy the functions:

1. Right-click on the **SticksAndStones.Functions** project and select **Publish**.
2. Select **Azure** as the destination for publishing and click **Next**:

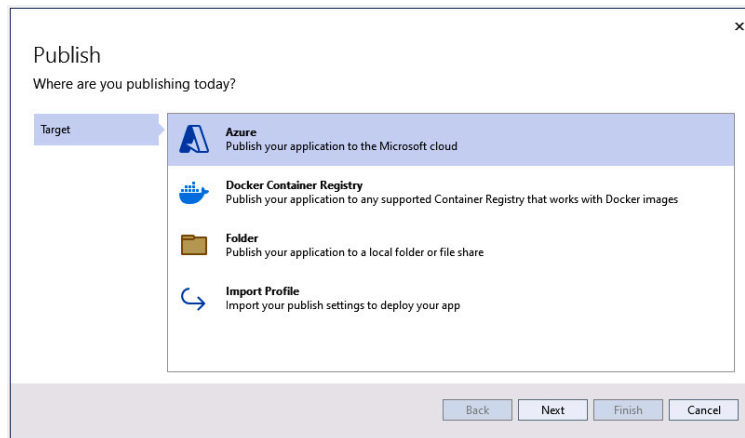


Figure 9.14 – Target selection when publishing

3. Choose **Azure Function App (Windows)** in the **Specific target** tab, then click **Next**:

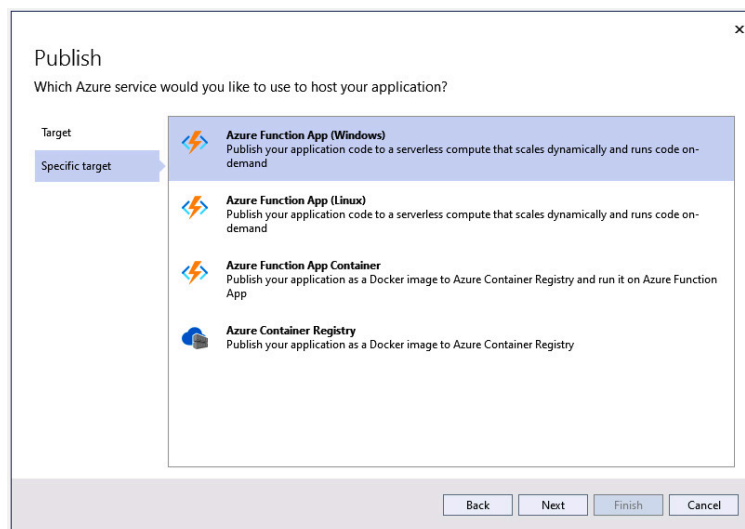


Figure 9.15 – Container selection when publishing

4. Sign in to the same Microsoft account that we used in the Azure portal when we were creating the **Function App** resource.
5. Select the subscription that contains the function app. All function apps we have in the subscription will now be loaded.
6. Select the function app and click **Finish**. If your app isn't showing up, click **Back** and choose the **Azure Function App (Linux)** option as you may not have changed the default when creating the service in the *Creating the Azure service for functions* section.
7. When the profile is created, click the **Publish** button.

The following screenshot shows the last step. After that, the publishing profile will be created:

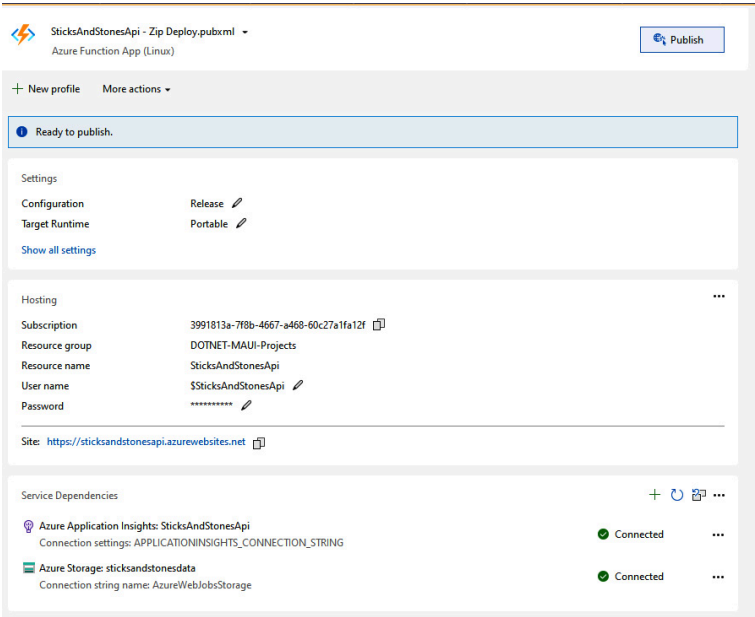


Figure 9.16 – Publishing Azure functions

Summary

In this chapter, we started by learning about a few Azure services, including SignalR, and Functions. Then, we created the services in Azure that our game server backend would need – a SignalR service for real-time communication, and finally, the Functions service to host our backend functions. After this, we implemented the Azure functions that would provide the functionality for our game.

We wrapped up this chapter by publishing our function code to the Azure Functions instance in Azure.

In the next chapter, we will build a game app that will use the backend we have built in this project.